

PROYECTO FINAL – API DE GESTIÓN DE TAREAS

Descripción general

Este proyecto consiste en una API RESTful desarrollada con Spring Boot, que permite gestionar tareas y usuarios mediante operaciones CRUD, con autenticación y autorización basada en roles (USER y ADMIN) utilizando JWT (JSON Web Token).

La aplicación se conecta a una base de datos MySQL y está documentada mediante Swagger UI para facilitar su prueba e interacción.

Tecnologías utilizadas

- Spring Boot 3.5.7
- Spring Data JPA
- Spring Security + JWT
- MySQL
- Swagger
- Java 21

Estructura del proyecto

- Controladores: Manejan las peticiones HTTP (*AuthController* y *TaskController*)
- Servicios: Implementan la lógica de negocio (*UserServiceImpl* y *TaskServiceImpl*)
- Repositorios: Interactúan con la base de datos mediante JPA (*UserRepo*, *TaskRepo*)
- Entidades: Modelan las tablas (*User*, *Role* y *Task*)
- Seguridad: Configuración de JWT y control de acceso (*JwtUtil*, *JwtAuthFilter* y *SecurityConfig*)
- Configuraciones adicionales: Swagger (*Swagger Config*) y variables de entorno en archivo *.env* para ocultar credenciales.

La Fig. 1 presenta la API implementada en Swagger.

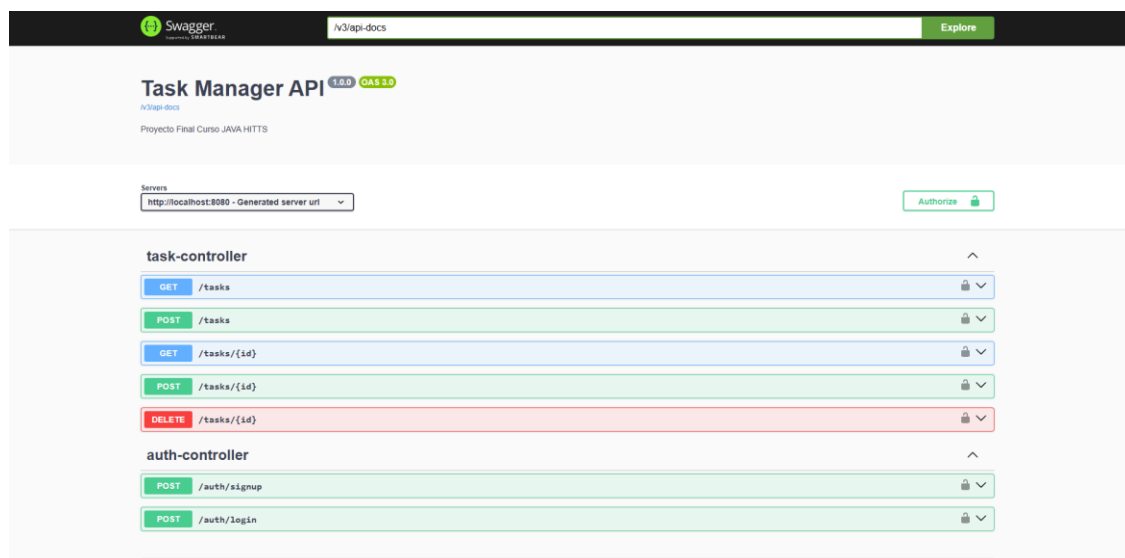


Fig. 1. API REST implementada en Swagger

La **Tabla 1** presenta el par de roles creados y sus distintos permisos.

Tabla 1. Roles y permisos para el API REST

Rol	Permisos
USER	Puede crear tareas y visualizar las propias
ADMIN	Puede realizar todas las operaciones CRUD sobre las tareas

Autenticación y autorización

1. Los usuarios se registran mediante el endpoint POST/auth/signup.
2. Luego inician sesión con POST/auth/login, obteniendo un token JWT.

POST

/auth/login

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "username": "edgarding",
  "password": "cruzazul"
}
```

Execute

Clear

Fig. 2. Inicio de sesión a través del endpoint mencionado

Responses

Curl

```
curl -X 'POST' \
'http://localhost:8080/auth/login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "username": "edgardrg",
  "password": "crutazul"
}'
```

Request URL

```
http://localhost:8080/auth/login
```

Server response

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6ImlhbmUiOmtCZWJJA0l0Yy9kCmZkdjZkdWtJoxNzYyNTQwZDdFQjFSM3BvYXBkaXkiOiJkbmMiL3BvdzVSIjE1fQCR0qGp0"</pre> Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Sun, 02 Nov 2025 05:00:10 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
200	OK	No links

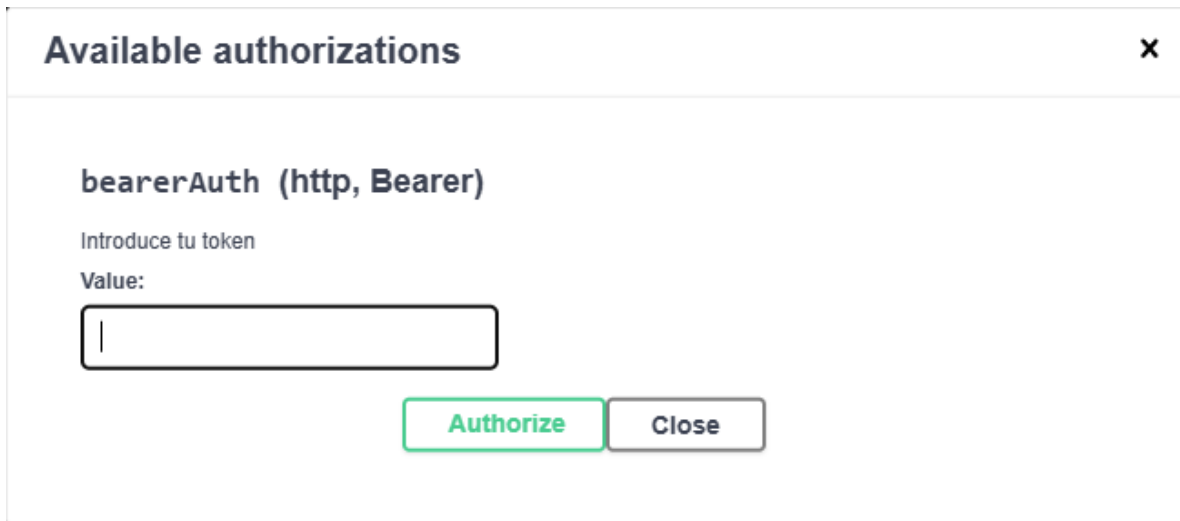
Media type

Controls Accept header

Example Value | Schema

Fig. 3. Token obtenido después de hacer el inicio de sesión.

3. El token se introduce en Swagger con el botón “*Authorize*” (candado).



Available authorizations ✕

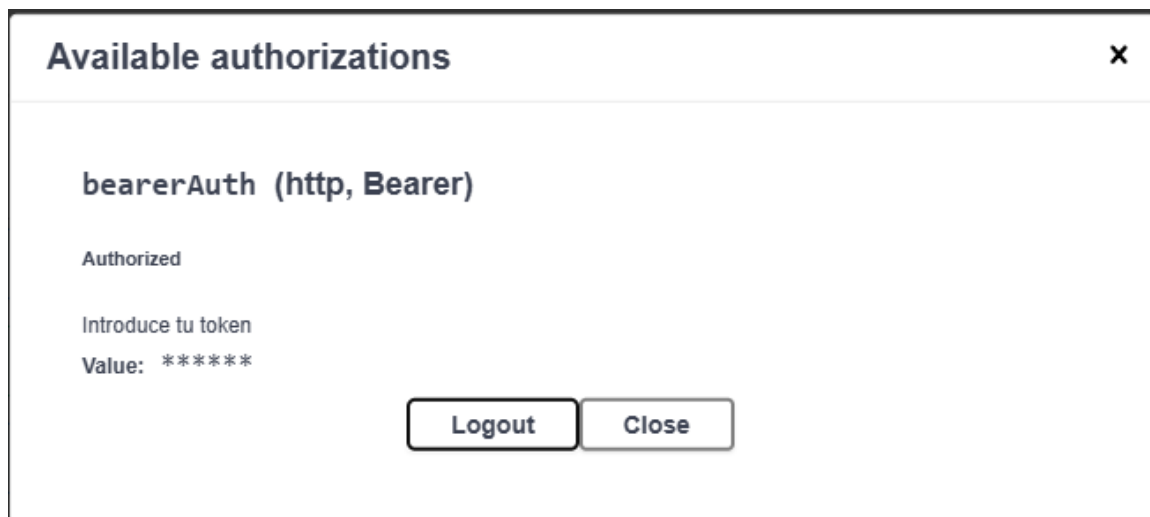
bearerAuth (http, Bearer)

Introduce tu token

Value:

Authorize **Close**

Fig. 4. Autorización en swagger.



Available authorizations ✕

bearerAuth (http, Bearer)

Authorized

Introduce tu token

Value: *****

Logout **Close**

Fig. 5. Token ingresado y autorizado.

4. Una vez autorizado, el usuario puede acceder a los endpoints protegidos según su rol.

Endpoints principales

La **Tabla 2** presenta los endpoints principales en esta API.

Tabla 2. Endpoints principales

Tipo	Ruta	Descripción
POST	/auth/signup	Registro de usuarios y administradores
POST	/auth/login	Autenticación y generación del token JWT

GET	/tasks	Listar tareas (todas si lo solicita el ADMIN, solo propias si es USER)
GET	/tasks/{id}	Consultar tarea por id
POST	/tasks	Crear una nueva tarea
POST	/tasks/{id}	Actualizar una tarea existente
DELETE	/tasks/{id}	Eliminar una tarea (solo ADMIN)

Las siguientes imágenes presentan los métodos para registrar tareas, listarlas, consultarlas por id y eliminar una tarea siendo administrador.

Registro de tarea:

POST /tasks

Parameters

No parameters

Request body required

application/json

```
{
  "id": 2,
  "title": "Hacer ejercicio",
  "description": "Rutina de pierna",
  "completed": false,
  "expires": "2025-11-03T05:02:26.600Z",
  "ownerId": 2
}
```

Execute Clear

Fig. 6. Endpoint POST para registrar una tarea

Borrar tarea como ADMIN:

DELETE /tasks/{id}

Parameters

Name Description

id required
integer(int64) 2

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8080/tasks/2' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/tasks/2
```

Server response

Code Details

204

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
date: Sun, 02 Nov 2025 05:38:27 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
x-content-type-options: nosniff
x-frame-options: DENY
x-ssr-protection: 0
```

Responses

Code Description Links

200 OK No links

Fig. 7. Endpoint DELETE para borrar la tarea previamente registrada

Registro de una nueva tarea

POST /tasks

Parameters

No parameters

Request body required

application/json

```
{  "id": 1,  "title": "Proyecto final",  "description": "Curso Java Hitts",  "completed": true,  "createdAt": "2025-11-03T05:11:17.257Z",  "dueDate": "2025-11-03T05:11:17.257Z",  "ownerId": 1}
```

Execute

Clear

Server response

Code

Details

201

Undocumented

Response body

```
{  "id": 1,  "title": "Proyecto final",  "description": "Curso Java Hitts",  "completed": false,  "createdAt": "2025-11-01T23:11:42.7662686",  "dueDate": "2025-11-03T05:11:17.257"} 
```

CopyDownload

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sun, 02 Nov 2025 05:11:42 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

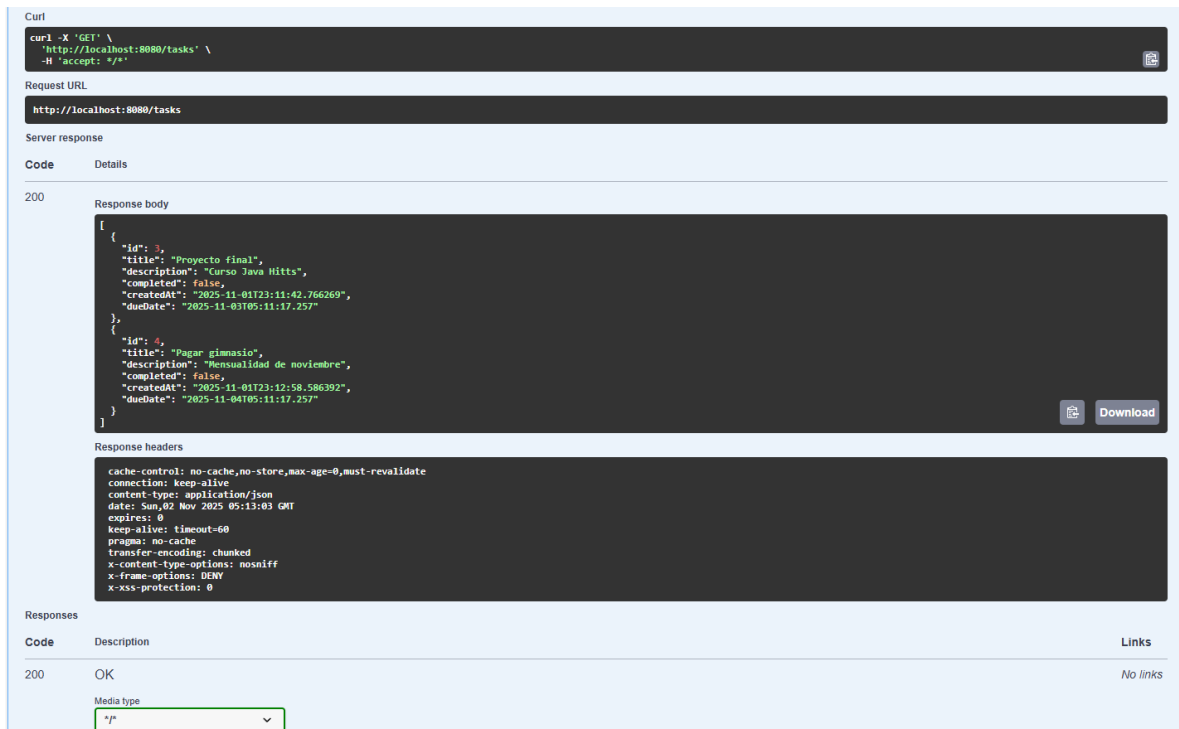
Responses

Code	Description	Links
200	OK	No links

Martin Turan

Fig. 8. Endpoint POST para registrar una tarea

Listar tareas



Curl

```
curl -X 'GET' \
  'http://localhost:8080/tasks' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/tasks

Server response

Code Details

200

Response body

```
{
  "id": 3,
  "title": "Proyecto final",
  "description": "Curso Java Hits",
  "completed": false,
  "createdAt": "2025-11-01T23:11:42.766269",
  "dueDate": "2025-11-03T05:11:17.257"
},
{
  "id": 4,
  "title": "Pagar gimnasio",
  "description": "Mensualidad de noviembre",
  "completed": false,
  "createdAt": "2025-11-01T23:12:58.586392",
  "dueDate": "2025-11-04T05:11:17.257"
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sun, 02 Nov 2025 05:13:03 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

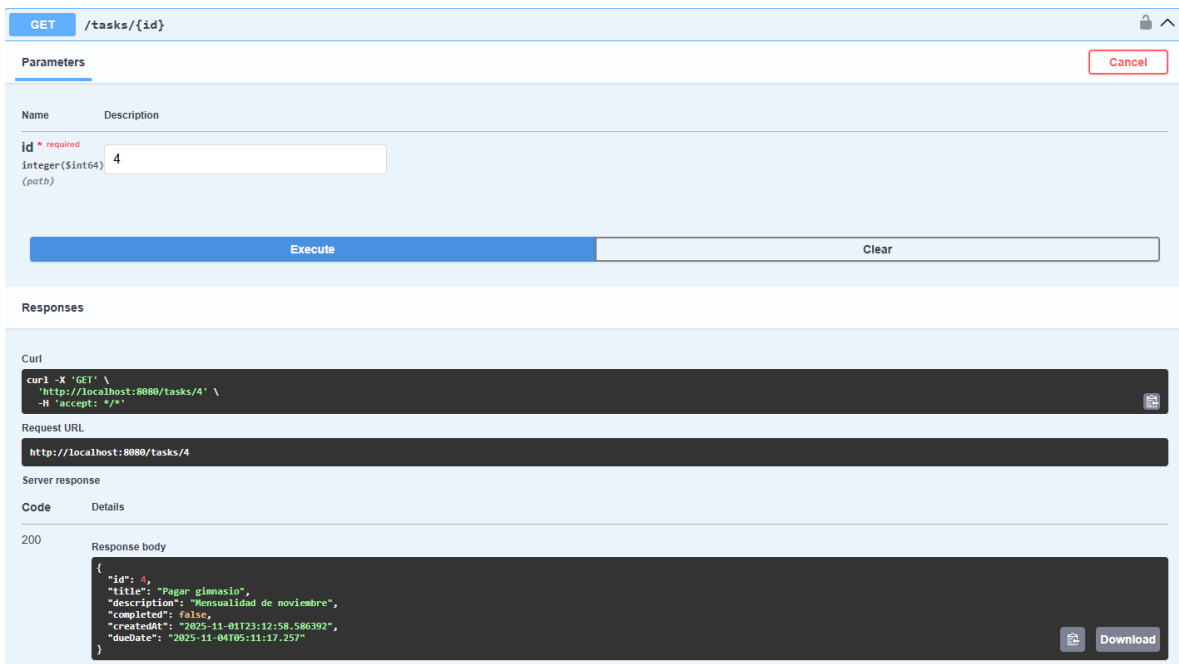
Code	Description	Links
200	OK	No links

Media type

/

Fig. 9. Endpoint GET para listar las tareas

Consultar tarea por id



GET /tasks/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int64)	4
(path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/tasks/4' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/tasks/4

Server response

Code Details

200

Response body

```
{
  "id": 4,
  "title": "Pagar gimnasio",
  "description": "Mensualidad de noviembre",
  "completed": false,
  "createdAt": "2025-11-01T23:12:58.586392",
  "dueDate": "2025-11-04T05:11:17.257"
}
```

Fig. 10. Endpoint GET para consultar una tarea a través de su id

Base de datos

La base de datos utilizada es MySQL, con las siguientes tablas principales:

- Users
- Roles
- User_roles
- Tasks

Las siguientes imágenes muestran la ejecución de comandos en MySQL Workbench para mostrar las tablas y su contenido.

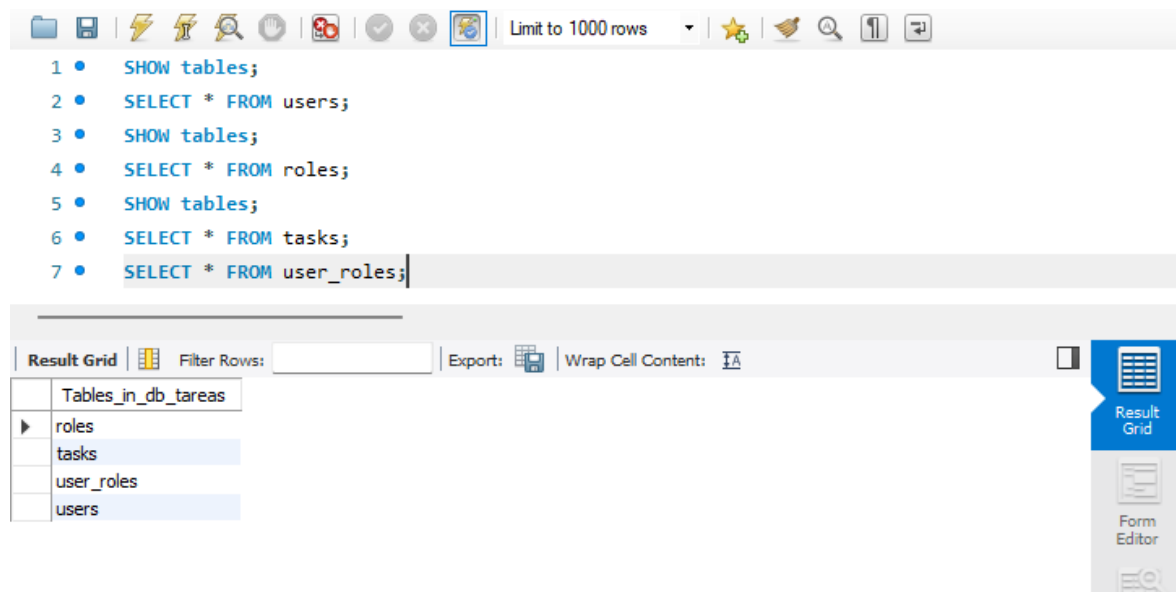


Fig. 11. Línea de comandos en MySQL Workbench

Contenido de la tabla *users*:

The screenshot shows the 'Result Grid' tab displaying the content of the 'users' table. The table has four columns: id, email, password, and username. The data is as follows:

id	email	password	username
1	admin@gmail.com	\$2a\$10\$E2LCia4eKis3M/CCrWS20tiEtm/yvV...	admin
2	edgareus23@gmail.com	\$2a\$10\$pX/yZeARpBCYCo5jAEVxZ.w8VBtFbID...	Edgard
7	edgard@email.com	\$2a\$10\$KYMciA8KdOnZzhUANCxhVerB.00FUELJ...	edgardrg
10	santi@gmail.com	\$2a\$10\$7hNHQdUhx5gPxYyuVTd2OsbhV7n9Z...	santiagoG
*	NULL	NULL	NULL

Fig. 12. Contenido de la tabla users

Contenido de la tabla *roles*:

Result Grid			Filter Rows:
	id	name	
▶	2	ROLE_ADMIN	
	1	ROLE_USER	
*	NULL	NULL	

Fig. 13. Contenido de la tabla roles

Contenido de la tabla *user_roles*

	user_id	role_id
▶	1	1
	10	1
	1	2
	2	2
	7	2
*	NULL	NULL

Fig. 14. Contenido de la tabla user_roles

Contenido de la tabla *tasks*

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	id	completed	created_at	description	due_date	title	user_id
▶	3	0	2025-11-01 23:11:42.766269	Curso Java Hitts	2025-11-03 05:11:17.257000	Proyecto final	1
	4	0	2025-11-01 23:12:58.586392	Mensualidad de noviembre	2025-11-04 05:11:17.257000	Pagar gimnasio	2
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 15. Contenido de la tabla tasks.

Conclusión

El proyecto cumple con los requisitos solicitados en las especificaciones, con excepción de la validación en Postman, debido a que Swagger UI permite realizar dicha validación de una forma más intuitiva y eficiente.

Esta APIREST reúne todos los conocimientos adquiridos en Java en estos tres meses de curso.